

Pattern-Guided Ontology Generation with Large Language Models^{*}

Samah Alkhuyaey^{1,*}, Terry R. Payne^{2,†}

¹Department of Data Science, Umm Al-Qura University, Makkah, 21955, Saudi Arabia

²Department of Computer Science, University of Liverpool, Liverpool, L69 3BX, UK

Abstract

Large Language Models (LLMs) are increasingly being applied across ontology engineering (OE) tasks, including conceptualisation, formalisation, and evaluation. However, despite substantial recent research, LLMs typically remain positioned as assistants to human ontology engineers, with persistent concerns regarding the quality, consistency, and methodological grounding of their outputs. Recent work has therefore explored grounding LLM-driven ontology generation in established OE methodologies and reusable modelling practices. Ontology Design Patterns (ODPs), which provide reusable solutions to recurrent ontology modelling problems, offer one potential mechanism for guiding LLMs towards more principled ontology construction. Yet, the effect of such pattern guidance on LLM-generated ontologies remains insufficiently understood. This paper investigates whether, and in what ways, ODP guidance affects the quality and modelling characteristics of LLM-generated ontologies. Given sets of Competency Questions (CQs), an LLM was provided with a library of Content ODPs and asked to identify relevant patterns and use them to inform ontology generation. The resulting ODP-guided ontologies were compared with baseline ontologies generated from the same CQs without pattern guidance in terms of structural characteristics, modelling pitfalls, and CQ answerability. In addition, both generation conditions were compared against corresponding reference ontologies to assess semantic and structural alignment. Results show that ODP guidance consistently produced larger and structurally more elaborate ontologies while reducing some modelling pitfalls. However, this increased structural richness did not consistently translate into improved CQ answerability or overall ontology quality. The findings indicate that ODP guidance primarily influenced how the LLM modelled domain knowledge, rather than consistently producing better ontologies, highlighting both the potential and limitations of ODPs as a mechanism for steering LLM-driven ontology generation.

Keywords

Ontology Engineering, Ontology Generation, Ontology Design Patterns, LLMs

1. Introduction

Ontology engineering (OE) provides systematic and principled approaches for developing, evaluating and maintaining ontologies while ensuring that domain knowledge is represented in a structured and reusable manner. The ontology development lifecycle encompasses activities such as requirements specification, conceptualisation, formalisation, evaluation, reuse, and maintenance. Despite the availability of mature methodologies, including METHONTOLOGY [1], NeOn [2], eXtreme Design (XD) [3], and Linked Open Terms (LOT) [4], as well as specialised ontology engineering tools, developing high-quality ontologies remains a cognitively demanding and resource-intensive activity. Effective ontology development requires not only domain expertise but also substantial knowledge of ontology modelling principles and knowledge representation languages such as OWL.

Recent literature has documented a rapid expansion in the application of Large Language Models (LLMs) across the ontology engineering lifecycle. LLMs have been investigated for tasks including requirements specification [5], conceptualisation [6, 7], taxonomy discovery [8], formalisation [7, 9], evaluation [10, 11], and documentation generation [6, 12], frequently assuming roles analogous to those

17th Workshop on Ontology Design and Patterns (WOP 2026) - Colocated with the 25th International Semantic Web Conference (ISWC 2026) October 25-29, 2026. Bari, Italy

^{*}Corresponding author.

[†]These authors contributed equally.

✉ smkhuyaey@uqu.edu.sa (S. Alkhuyaey); T.R.Payne@liverpool.ac.uk (T. R. Payne)

ORCID 0000-0001-8883-1172 (S. Alkhuyaey); 0000-0002-0106-8731 (T. R. Payne)



© 2026 Copyright for this paper by its authors. Use permitted under Creative Commons License Attribution 4.0 International (CC BY 4.0).

of ontology engineers and domain experts. These approaches operate on heterogeneous inputs, including text, competency questions (CQs), ontology fragments, RDF/OWL artefacts, and existing taxonomies, to produce outputs such as ontology axioms, conceptual structures and documentation [13]. The ability of LLMs to interpret natural-language requirements and generate structured knowledge representations offers significant potential to reduce the manual effort required for ontology development.

Nevertheless, current evidence suggests that LLMs cannot yet reliably replace human ontology engineers in autonomous ontology development workflows. Existing studies have reported persistent difficulties in constructing sufficiently deep and coherent ontology hierarchies, maintaining logical consistency, and representing highly specialised domains [6, 14, 15, 16]. LLM-generated ontologies may also exhibit hallucinations, incomplete conceptualisations, modelling inconsistencies, and substantial variability resulting from prompt sensitivity [17, 18, 19]. Consequently, the literature predominantly positions LLMs as assistants rather than autonomous ontology engineers, with human expertise remaining necessary to validate conceptualisations, correct logical errors and ensure alignment with domain requirements [10, 12, 14, 20]. LLMs may alleviate aspects of the knowledge-acquisition bottleneck when appropriately guided, but their effectiveness depends substantially on how ontology engineering tasks are formulated, structured, and constrained [8].

These limitations have motivated growing interest in mechanisms for providing LLMs with stronger methodological and modelling guidance. Rather than relying solely on generating ontologies from ontology requirements (such as competency questions), recent work has explored the decomposition of ontology engineering into smaller, modular activities and embedding LLMs within structured workflows informed by established ontology engineering practices [17, 19]. This perspective is particularly relevant to ontology generation, where the same domain requirement may admit multiple valid modelling realisations, yet unconstrained LLMs may produce representations that are incomplete, inconsistent, or insufficiently explicit [8, 21, 22]. The central challenge is therefore not merely whether an LLM can generate an ontology, but whether established modelling knowledge can effectively guide the generation process towards more principled and higher-quality representations.

Ontology Design Patterns (ODPs) provide a particularly relevant form of such modelling guidance. ODPs capture reusable solutions to recurrent ontology modelling problems and are intended to promote reuse, modularity, and consistency [23, 24]. By providing established conceptual structures that can be adapted and composed according to domain requirements, ODPs could potentially serve as scaffolding for LLM-driven ontology generation. Rather than requiring the LLM to derive every modelling decision directly from ontology requirements, pattern guidance can expose the model to reusable structures for representing recurring modelling scenarios. However, whether incorporating ODPs into LLM-driven ontology generation actually leads to better ontologies remains insufficiently understood.

We therefore investigate the effect of ODP guidance on LLM-generated ontologies. Given sets of CQs, an LLM is provided with a number of Content ODPs from which relevant patterns may be selected and subsequently used to guide ontology generation. The resulting ODP-guided ontologies are compared with baseline ontologies generated from the same CQs without pattern guidance. The two generation conditions are evaluated comparatively in terms of their structural characteristics, modelling pitfalls, and CQ answerability, while their alignment with corresponding reference ontologies is assessed separately using semantic similarity metrics. Accordingly, this study addresses the following research questions:

- Does ODP-guided generation improve the quality of LLM-generated ontologies?
- How does ODP-guided ontology generation affect the types of modelling errors that are introduced, reduced or amplified?

The remainder of the paper is structured as follows: Section 2 situates the work within the relevant background and related literature. Section 3 sets out the methodological design and evaluation framework adopted in the study. The resulting findings and their implications are examined in Section 4, before Sections 5 and 6 draw together the main conclusions of this work.

2. Background and Related Work

2.1. Ontology Engineering (OE)

Ontology Engineering (OE) provides the methodologies, principles, and tools required to develop domain and application-specific ontological models. Contemporary ontology engineering methodologies structure the development process into a series of interconnected phases, including requirements specification, conceptualisation, implementation, publication, and maintenance. Established methodologies such as METHONTOLOGY, NeOn, and Linked Open Terms (LOT) offer systematic guidance across these phases, supporting the development of reusable and maintainable ontologies [1, 2, 4]. Despite significant methodological advances, ontology engineering remains a knowledge-intensive activity that requires both domain expertise and proficiency in formal knowledge representation. A persistent challenge concerns the transformation of requirements into appropriate conceptual and logical structures. While ontology engineering methodologies provide procedural guidance, they offer limited support for selecting among alternative modelling solutions. In practice, identical requirements can often be represented through multiple valid conceptualisations, producing ontologies that are logically sound yet differ substantially in terms of maintainability, interoperability, and reuse potential. Consequently, recent research has increasingly emphasised reuse-oriented approaches that seek to reduce modelling variability and promote greater consistency in ontology design [25].

2.2. Ontology Design Patterns (ODPs)

Ontology Design Patterns (ODPs) emerged as a response to the recurring modelling challenges encountered during ontology development. ODPs have been defined as reusable solutions to recurrent ontology design problems, encapsulating modelling expertise in a form that can be systematically reused across domains [26, 27]. The central premise underlying ODPs is that many ontology engineering problems are not unique but rather manifestations of recurring conceptual structures. Instead of developing ontologies from first principles, engineers can draw upon established modelling solutions that capture proven design choices. This perspective was formalised through comprehensive ODP classifications encompassing content, logical, reasoning, correspondence, and architectural patterns. ODPs therefore function not only as reusable ontology fragments but also as explicit representations of modelling knowledge and design rationale. Empirical evidence supports the benefits of pattern-based ontology development. Indeed, the growing recognition of ODPs as reusable solutions to recurring modelling problems directly inspired the development of the eXtreme Design (XD) [3] methodology, a reuse-oriented ontology engineering approach centred on the systematic identification, selection, and composition of ontology design patterns. Studies conducted within the XD methodology demonstrate that Content ODPs, when combined with methodological and tool support, improve ontology quality and substantially reduce common modelling errors through iterative testing and validation workflows [28]. However, despite these advantages, investigations of widely used biomedical ontologies indicate that explicit ODP adoption remains relatively limited. This suggests that the identification and selection of appropriate patterns continues to represent a significant barrier to broader pattern-based ontology engineering [29].

2.3. Ontology Engineering with LLMs

The emergence of LLMs has stimulated considerable interest in their use in ontology engineering, with a rapidly expanding range of applications. In their literature survey, Garijo et al. [13] map existing approaches to ontology engineering activities and report the use of LLMs for competency question generation, ontology learning, ontology matching, ontology evaluation, and ontology documentation, amongst other applications. Their analysis further reveals that research has concentrated predominantly on conceptualisation and encoding activities, whereas ontology reuse, selection, and adaptation remain comparatively underexplored. Building on this work, Li et al.'s [16] systematic review of thirty studies shows that LLMs have been employed across ontology requirements specification, implementation, publication, and maintenance, acting variously as ontology engineers, domain experts, and evaluators.

At the requirements stage, LLMs have been employed to support the elicitation and formalisation of ontology requirements. For example, Zhang et al. [10] proposed *OntoChat*, a conversational ontology engineering framework that assists stakeholders and ontology engineers in generating user stories, extracting competency questions, analysing requirements, and testing preliminary ontologies. Their results suggest that LLM-driven conversational workflows can reduce the effort associated with requirements engineering and improve stakeholder engagement.

At the conceptualisation and implementation stages, several studies have explored using LLMs to generate ontological artefacts directly from ontology requirements. Mateiu and Groza [30] developed a fine-tuned GPT-3 model that translated natural-language statements into OWL Functional Syntax, demonstrating significantly better performance than zero-shot and few-shot prompting, which frequently produced inconsistent or incorrect axioms. Similarly, Giglou et al. [8] evaluated several LLM families across ontology learning tasks, including term typing, taxonomy discovery, and relation extraction. While LLMs were able to identify concepts and relationships from text, performance varied across tasks and domains, indicating that foundational models still lack the reasoning and domain expertise required for reliable ontology construction. At later stages of ontology development, Toro et al. [6] applied LLMs to automatically generate ontology documentation from OWL ontologies. Their results showed that the generated documentation improved accessibility and was generally perceived as useful, but inconsistencies in completeness and semantic accuracy highlighted the need for expert review.

Although LLMs have become increasingly useful in ontology engineering, the literature consistently concludes that they complement rather than replace ontology engineers. Their strengths lie in accelerating knowledge-intensive tasks and generating candidate artefacts, while their limitations stem from insufficient reasoning, domain expertise, and modelling judgement. Studies by Giglou et al. [8], Li et al. [16] and Bakker et al. [31] all emphasise that expert validation and human involvement remain essential throughout the ontology engineering lifecycle. Other studies point to more fundamental challenges associated with the use of LLMs for knowledge representation tasks. Garijo et al. [13] identify persistent concerns relating to limited reasoning capabilities, hallucinations and opacity, all of which can affect the quality and reliability of generated ontological artefacts. Empirical investigations of collaborative ontology engineering workflows reinforce these concerns. Doumanas et al. [15] report that, although increasing LLM involvement substantially accelerates ontology development, ontology quality improves when human participation remains high. Likewise, Lippolis et al. [19] observe considerable variability in the quality of ontologies generated from competency questions and user stories, identifying recurring modelling mistakes even in configurations that outperform novice ontology engineers.

Taken together, these findings suggest that LLMs are currently most effective when supporting specific, well-defined ontology engineering tasks rather than performing autonomous ontology construction. Accordingly, several studies advocate integrating LLMs into structured development processes with methodological guidance and expert oversight, where they assist rather than replace ontology engineers.

2.4. Pattern-Guided Ontology Engineering with LLMs

Current LLM-based ontology engineering approaches primarily focus on generating ontological artefacts directly from competency questions (CQs), user stories, or other natural-language requirements with limited structural guidance [19]. At the same time, OE surveys emphasise the lack of approaches for LLM-based ontology reuse and selection [13]. As ODPs were originally conceived as explicit representations of modelling knowledge, their value extends beyond the ontology fragments they provide to encompass the design rationale and modelling decisions they encode. Consequently, integrating ODPs into LLM-assisted ontology engineering workflows offers a promising mechanism for introducing methodological guidance and reuse-oriented design principles into ontology generation processes. One of the first attempts to explicitly examine the role of ODPs in LLM-assisted ontology engineering was *OntoGenia* [19]. Grounded in the *eXtreme Design* methodology, *OntoGenia* incorporates expert-selected Content ODPs into a metacognitive prompting strategy to guide ontology specification and formalisation from CQs. In their evaluation, ontology experts selected eight Content ODPs relevant to the *African Wildlife* domain, including *AgentRole*, *Classification*, *PartOf*, *Linnaean Taxonomy*, and *SpeciesEat*. These

patterns were dynamically incorporated into prompts to provide structured modelling knowledge during ontology generation. Results showed that combining ODPs with metacognitive prompting generally produced richer ontology formalisations, with higher numbers of axioms, logical axioms, classes, and properties than approaches based solely on CQs. However, their framework relies on manually selected ODPs rather than tasking the LLM with identifying appropriate patterns, and its evaluation primarily focuses on ontology size and structural characteristics, leaving open the question of whether ODP guidance improves ontology quality with regard to other dimensions.

More recently, Shyama et al. [17] explored ODP-guided prompting in the related task of ontology population. Rather than generating ontology schemas, the authors used ODPs to guide LLMs in instantiating existing ontologies, motivated by observations that conventional prompting often failed to capture hierarchical relationships and design patterns embedded within ontologies. Their pattern-based prompting strategy substantially reduced hallucinations and markedly improved the generation of complex relation assertions compared with conventional few-shot prompting. Although this work targets ontology population rather than ontology generation, it provides further evidence that explicitly exposing LLMs to ontology design patterns can improve modelling performance when ontology structure plays a central role.

3. Methodology

The objective of this study is to investigate the effect of Ontology Design Patterns (ODPs) on LLM-based ontology generation from CQs. Specifically, the study examines whether explicitly exposing an LLM to a library of reusable Content ODPs during ontology generation improves the quality of the resulting ontologies compared with conventional CQ-based generation. To this end, a controlled comparative experiment was conducted under two prompting conditions. In the baseline condition, the LLM generated ontologies solely from expert-authored CQs. In the ODP-guided condition, the same CQs were accompanied by a curated library of Content ODPs represented as OWL ontology fragments. The model was instructed to consult this library during ontology generation and to incorporate one or more ODPs only where they provided an appropriate modelling solution, rather than assuming that every CQ required a pattern. The resulting ontologies were evaluated using complementary quality assessment strategies. Structural characteristics, ontology pitfalls, and CQ answerability were compared directly between the baseline and ODP-guided ontologies to assess the effect of ODP-guided generation. In addition, semantic similarity measures were computed by comparing each generated ontology against its corresponding expert-developed reference ontology, enabling an assessment of the extent to which each generation strategy reproduced the intended conceptual model.

3.1. Experimental Materials

3.1.1. Ontology Design Pattern Library

A prerequisite for the experiment was the construction of a curated ODP library suitable for use by an LLM during ontology generation. Although numerous ODP collections have been proposed over the past two decades, they exhibit considerable variation in provenance, documentation quality, modelling granularity, and intended purpose. Consequently, a systematic selection process was undertaken to construct a coherent pattern library for the experiment. As this study represents an initial investigation of ODP-guided ontology generation, the library was restricted to Content ODPs. Content ODPs were selected because they capture recurring domain modelling solutions and are intended to support the conceptual modelling decisions required during ontology development. This contrasts with other categories of ODPs, such as Logical, Architectural, or Correspondence patterns, that address different aspects of ontology engineering, and were therefore considered outside the scope of this study.

The selection process was domain-independent and based on a predefined set of inclusion and exclusion criteria designed to maximise the consistency and interpretability of the library. First, only patterns explicitly classified as Content ODPs were considered. Second, each pattern was required to represent a self-contained and reusable ontology module, rather than a modelling guideline or

methodological recommendation. Third, only well-documented patterns were retained. Previous studies have highlighted substantial variability in ODP documentation, with some patterns providing comprehensive descriptions of their modelling intent and implementation, while others contain only minimal or incomplete documentation. Since the patterns were intended for consumption by an LLM rather than a human ontology engineer, sufficient semantic context was essential to enable reliable interpretation of each pattern’s intended use. Consequently, patterns lacking a clearly stated modelling intent, accompanying documentation, or a corresponding OWL implementation were excluded. Similarly, deprecated, incomplete, or duplicate patterns were removed from the final library.

The resulting library comprised 80 Content ODPs, spanning both generic and domain-specific modelling problems. For each pattern, the original OWL implementation was included as an ontology fragment, preserving the formal axioms and modelling structures intended by the pattern designers. These OWL fragments formed the ODP knowledge base supplied to the LLM during the ODP-guided generation condition. The complete library is publicly available in the accompanying *GitHub*¹ repository.

3.1.2. Competency Questions

To evaluate ODP-guided ontology generation, we employed an established benchmark comprising expert-authored CQs together with their corresponding reference ontologies. Three ontologies were selected: the *African Wildlife Ontology (AWO)*, the *Stuff Ontology* [32], and the *Wine Ontology*.² These ontologies were chosen because they represent diverse modelling domains and conceptual structures, while providing publicly available CQs and expert-developed ontology implementations. This benchmark is particularly well suited to the present study because it provides both the requirements specification, in the form of CQs, and the corresponding reference ontologies that realise those requirements. During ontology generation, the LLM was provided only with the CQs; the reference ontologies were withheld and used exclusively for the second part of the evaluation. This ensured that the generation process was driven solely by the requirements expressed in the CQs, while enabling subsequent comparison with expert-developed ontologies where appropriate.

3.1.3. LLM and Prompt Design

The experiment was conducted using GPT-5.5 between 5–7 July 2026. A temperature of 0 was used, while the model’s default maximum output limit of 128,000 tokens was retained. To isolate the effect of ODPs on ontology generation, two prompting conditions were evaluated under identical model and generation settings. In the baseline condition, the model was provided only with the CQs for a given domain. It was instructed to assume the role of an ontology engineer and to analyse each CQ, derive the required ontology elements, and produce the corresponding ontology axioms. After processing all CQs, the model was instructed to consolidate the generated axioms into a single coherent OWL ontology. In the ODP-guided condition, the model received the same CQs together with the curated ODP library described in the previous section, comprising 80 Content ODP fragments and approximately 870,000 characters (over 200,000 tokens). The prompt first introduced the concept of ODPs before providing access to the pattern library. For each CQ, the model was instructed to determine whether one or more ODPs from the library would provide an appropriate modelling solution. To discourage unnecessary pattern usage, the prompt explicitly stated that no ODP should be selected if the CQ could be adequately modelled using standard ontology constructs. When an appropriate ODP was identified, the model was instructed to generate ontology axioms that incorporated the modelling structures and ontology elements provided by the selected pattern(s). Otherwise, the model generated the required axioms without relying on an ODP. As in the baseline condition, the axioms produced for all CQs were subsequently integrated into a single coherent ontology. For each benchmark domain, two ontologies were therefore produced: a baseline ontology generated from CQs alone (e.g., *Stuff_{baseline}*) and an ODP-guided ontology generated using the same CQs supplemented with the ODP library (e.g., *Stuff_{ODP}*).

¹https://github.com/smskshk/odp_guided_ontology_generation

²<https://github.com/UCDavisLibrary/wine-ontology/blob/master/wine-ontology.owl>

3.2. Evaluation

The generated ontologies were assessed using a complementary set of structural, syntactic, semantic, and competency-based evaluation measures. Together, these evaluations examine not only whether ODP-guided prompting produces larger or more expressive ontologies, but also whether it leads to improved modelling quality, better satisfaction of the ontology requirements, and greater agreement with expert-developed ontologies. First, the structural characteristics of the baseline and ODP-guided ontologies were compared. Structural analysis considered ontology size and complexity, including the number of classes, properties, subclass relations, logical axioms, and other constructs. The objective of this analysis is to determine whether access to ODPs influences the overall modelling structure adopted by the LLM and to identify differences in modelling complexity between the two prompting conditions. Second, ontology quality was assessed using the *OOPS!* (OntOlogy Pitfall Scanner!)³ framework, which automatically detects common ontology modelling pitfalls [33]. The analysis compares both the number and types of pitfalls identified in the baseline and ODP-guided ontologies. Since ODPs are intended to promote established ontology engineering best practices, this evaluation examines whether explicit ODP guidance reduces modelling errors or instead introduces new categories of pitfalls. Third, the generated ontologies were evaluated with respect to CQ answerability. This assessment determines whether the generated ontologies are capable of answering the CQs through the execution of expert-authored SPARQL queries developed for the corresponding reference ontologies⁴. CQ answerability provides a task-based evaluation of ontology quality by measuring the extent to which the generated ontology fulfils its intended requirements specification rather than merely exhibiting desirable structural properties. Prior to query execution, each ontology was classified using the *HermiT* reasoner in *Protégé*, and the inferred ontology was exported. The SPARQL queries were then executed against the inferred ontology to evaluate CQ answerability. Finally, the generated ontologies were compared with their corresponding reference ontologies using two semantic similarity measures: cosine and Jaccard similarity. Unlike the previous evaluations, which compare the two generation conditions directly, this analysis evaluates each generated ontology independently against an expert-developed ontology constructed from the same CQs. The reference ontologies, therefore, serve as proxies for expert modelling decisions. Semantic similarity provides insight into the extent to which the LLM reproduces the conceptual structures and modelling choices adopted by ontology engineers, despite potentially employing different modelling strategies. Jaccard similarity quantifies overlap in the modelling constructs and ontology elements used, whereas cosine similarity captures broader semantic correspondence between the ontologies even when equivalent knowledge is represented using different modelling structures. This distinction is particularly important in ontology engineering, where multiple valid modelling solutions may satisfy the same CQs while differing in their formal representation.

4. Results and Discussion

A total of six ontologies were generated, comprising three baseline ontologies generated solely from CQs and three ODP-guided ontologies generated from the same CQs together with the Content ODP library. Table 1 summarises the descriptive characteristics of the generated ontologies.

Across all three case studies, ODP-guided generation consistently produced larger and structurally richer ontologies than the CQ-only baseline. The increase is evident in the total number of axioms, logical axioms, classes and properties. *Wine* exhibited the largest increase in total axioms, while *AWO* experienced the largest proportional increase in logical axioms, suggesting that ODP guidance in *AWO* primarily enriched the ontology’s logical structure rather than merely increasing its overall size. This is consistent with the findings of Lippolis et al. [19], where the incorporation of ODPs similarly increased the number of classes, object properties, and axioms, suggesting that pattern guidance tends to promote more structurally elaborate ontology representations. The increase in ontology size was accompanied

³<https://oops.linkeddata.es/index.jsp>

⁴As the *Wine Ontology* benchmark does not provide corresponding SPARQL queries, this evaluation was performed only for the *Stuff* and *AWO* ontologies.

Table 1

Descriptive statistics of the generated ontologies. Main counts exclude axioms and entities introduced through resolved `owl:imports`; imported content is reported separately.

Metric	Stuff _{baseline}	Stuff _{ODP}	AWO _{baseline}	AWO _{ODP}	Wine _{baseline}	Wine _{ODP}
Axioms	134	176	103	141	117	206
Logical Axioms	72	82	49	86	66	77
Class count	20	33	16	33	20	36
Object Property count	13	15	11	15	13	14
Data Property count	2	0	0	1	1	1
Individual count	10	0	0	1	10	12
SubClassOf	23	33	13	13	21	26
Equivalent Classes	1	3	8	21	3	8
Disjoint Classes	3	6	4	5	1	2
SubObjectPropertyOf	3	5	2	3	0	5
Inverse Object Property	2	4	3	6	0	6
Transitive	2	0	0	4	0	0
Object Property Domain	13	15	9	15	13	14
Object Property Range	13	15	9	15	13	14
Data Property Domain	1	0	0	1	1	1
Data Property Range	2	0	0	1	1	1
Imported axioms	NA	0	NA	0	NA	117

by a substantially richer logical structure. ODP-guided ontologies contained more equivalent class definitions, disjointness axioms and inverse object properties. For example, the number of equivalent class axioms increased from 1 to 3 in *Stuff*, from 8 to 21 in *AWO* and from 3 to 8 in *Wine*. Likewise, inverse object properties approximately doubled across all three domains. Object-property modelling also became more explicit as every ODP-guided ontology contained additional domain and range declarations compared with the baseline. This reflects the tendency of the selected ODPs to introduce well-defined object-property signatures as part of their reusable modelling structures. However, the use of datatype properties did not exhibit the same trend. This may be because the CQs did not explicitly elicit modelling requirements involving literal values or data attributes and therefore provided little impetus for either generation approach to introduce datatype properties.

This descriptive analysis demonstrates that the availability of ODPs considerably influences the modelling behaviour of the language model. Rather than simply generating larger ontologies, the LLM systematically produced ontologies with more expressive logical structures (as can be seen from the logical axiom counts in Table 1). One notable observation is the increase in equivalent class definitions, which generally represent stronger semantic commitments than simple subclass assertions because they define necessary and sufficient conditions for class membership. The large increase observed in *AWO* (from 8 to 21) and *Wine* (from 3 to 8) suggests that ODP guidance encouraged the model to formulate richer class definitions derived from existential restrictions, classifications and contextual modelling rather than relying solely on taxonomic hierarchies. Overall, these findings suggest that ODPs primarily influence how the LLM models domain knowledge. Whether these additional modelling commitments ultimately improve ontology quality is examined through the subsequent evaluation metrics.

The generated ontologies also differed in how the LLM operationalised ODP guidance. In *Wine*, it reused three patterns through resolvable `owl:imports`; in *Stuff*, it produced two import declarations targeting non-resolving `example.org` IRIs; and in *AWO*, it authored adapted pattern axioms directly within the ontology and documented their provenance through annotations. This variation indicates that the LLM did not apply a consistent strategy for pattern reuse across the three domains. A possible explanation is that, although the prompt instructed the model to incorporate relevant ODPs, it did not prescribe *how* they should be incorporated (e.g. through direct `owl:imports`, inline adaptation of pattern axioms, or other forms of reuse), leaving the implementation strategy to the model. Notably, these differences emerged despite generation being performed with a temperature of zero, suggesting that de-

Table 2

Results of the OOPS! pitfall scanner for each generated ontology.

Pitfall	Description	Stuff baseline	Stuff ODP	AWO baseline	AWO ODP	Wine baseline	Wine ODP
P02	Creating synonyms as classes	0	0	0	1	0	0
P04	Creating an unconnected ontology element	0	1	1	0	0	1
P07	Merging different concepts in the same class	0	0	1	4	0	0
P11	Missing domain or range in properties	3	0	3	0	0	0
P13	Inverse properties are not explicitly declared	9	6	5	2	13	0
P29	Defining wrong transitive relations	0	0	0	2	0	0

terministic decoding alone does not ensure consistent ontology engineering behaviour. This observation is consistent with previous studies on LLM-assisted ontology engineering, which have similarly reported variation in the modelling decisions and ontology structures produced from comparable inputs [9].

To assess the logical consistency of the generated ontologies, all six generated ontologies were classified using *Hermit*.⁵ Four ontologies were found to be logically consistent, whereas both *AWO* ontologies contained a single unsatisfiable class. In the baseline *AWO*, *CarnivorousPlant* was unsatisfiable because the inverse property declaration `isEatenBy inverseOf eats`, together with the range restriction on `isEatenBy`, implicitly entailed *Animal* as the domain of `eats`. As a result, *CarnivorousPlant* was inferred to be an *Animal* while simultaneously being asserted as a subclass of *Plant*, thus contradicting the declared disjointness between *Animal* and *Plant*. In the ODP-guided ontology, *CarnivoreThatEatsPlantMaterial* was unsatisfiable due to conflicting existential and universal restrictions. The inherited definition of *Carnivore* required all fillers of `eats` to belong to *Animal*, while the additional restriction $\exists \text{ eats.PlantOrPlantPart}$ required the existence of a filler belonging to *Plant* or *PlantPart*. Since *Animal*, *Plant*, and *PlantPart* were declared mutually disjoint, the class became logically unsatisfiable. Notably, this inconsistency is a result of the composition of two individually reasonable modelling restrictions that became incompatible when combined. This illustrates how correct local modelling decisions can interact to produce unintended logical consequences that are only revealed through reasoning. Overall, the reasoning analysis indicates that logical inconsistencies were limited, with the detected inconsistencies arising from subtle interactions between OWL semantics rather than widespread logical defects.

Table 2 presents the ontology engineering pitfalls identified by OOPS! across the six generated ontologies. Overall, ODP-guided generation did not consistently reduce the total number of ontology pitfalls; instead, it altered their nature. Across the three case studies, ODP-guided ontologies generally exhibited fewer structural omissions, particularly missing property domains/ranges (P11) and undeclared inverse properties (P13), while occasionally introducing more expressive but conceptually incorrect modelling decisions. The clearest improvement was observed in the explicit specification of object properties. Missing domain and range declarations (P11), present in both *Stuff* and *AWO* baselines, were completely eliminated in their ODP-guided counterparts. This is consistent with previous literature on LLM-driven ontology generation, which has identified incomplete property modelling, particularly the omission of domain and range declarations, as a recurrent limitation of LLM-generated ontologies [14, 18]. Similarly, the number of missing inverse-property declarations (P13) decreased across all three domains, from 9 to 6 for *Stuff*, from 5 to 2 for *AWO*, and from 13 to none for *Wine*. This trend is consistent with the modelling commitments prescribed by several of the selected ODPs, such as *Classification*, *Componentency*, *SpeciesEat* and *SpeciesHabitat*, which explicitly introduce inverse property pairs and well-defined property signatures. For example, the *Componentency* ODP generated `hasSubstuff` together with its inverse `isSubstuffOf`, while the *Classification* ODP introduced `categorisedAs` as a specialisation of `isClassifiedBy`, both with explicit domains and ranges.

Meanwhile, ODP guidance introduced new modelling issues that were largely absent from the baseline ontologies. *AWO* exhibited the greatest increase in conceptual modelling errors, including synonym

⁵Both *Wine* ontologies required replacing `xsd:gYear` with an OWL 2 DL-compliant datatype prior to reasoning, as `xsd:gYear` is not included in the OWL 2 datatype map. This was treated as a standards-conformance issue rather than a modelling error.

classes (P02), concept conflation (P07), and incorrect use of transitive properties (P29). An examination of the generated ontology indicates that these issues stem from the LLM synthesising convenience classes (e.g. *PlantOrPlantPart* and *HerbivorousOrOmnivorousAnimal*). This suggests that, while ODPs encourage richer conceptual modelling, they also increase the likelihood of inappropriate abstraction and misuse of OWL semantics. When inspecting the sources of such pitfalls, it was noted that several selected ODPs encouraged the LLM to create additional modelling artefacts that subsequently triggered OOPS! warnings. For example, the *Region* ODP in *Wine* introduced classes such as *WineCharacteristic* and *WineCharacteristicDimension* to represent wine attributes as first-class entities rather than literals. Thus, some OOPS! findings appear to be a consequence of partially realised ODP implementations rather than incorrect ODP selection. The most interesting relationship is where the selected ODP appears to have been interpreted incorrectly. Several ecological CQs generated convenience classes such as *PlantOrPlantPart*, *AnimalThatEatsPlantsOrPlantParts*, and *HerbivorousOrOmnivorousAnimal*. These classes were introduced while instantiating *SpeciesEat* and *PartOf* to satisfy multiple CQs simultaneously, and they correspond closely to the increase in P07 (“Merging different concepts in the same class”). Rather than reflecting an incorrect ODP choice, these errors indicate that the LLM tended to overgeneralise concepts when composing multiple patterns.

Taken together, these results suggest that ODP guidance primarily shifts modelling errors rather than eliminating them. Baseline ontologies were characterised by omissions in property specification, whereas ODP-guided ontologies tended to exhibit fewer structural omissions but more sophisticated conceptual modelling errors arising from the application and composition of reusable patterns. This behaviour reflects a trade-off between modelling completeness and modelling complexity: ODPs encourage richer and more explicit ontological representations, but they also require the LLM to make stronger modelling commitments that seem forced, or artificially constructed, to satisfy the CQ, thereby increasing the risk of introducing logically or conceptually inappropriate constructs.

The preceding analyses focused on the structural and engineering characteristics of the generated ontologies. The following evaluation examines their semantic adequacy through two complementary dimensions: 1) alignment with the corresponding reference ontologies, measured using cosine and Jaccard similarity; and 2) competency-question answerability, which assesses whether the generated ontologies satisfy the intended knowledge requirements. It should be noted that the reference ontologies were not assumed to have been developed using ODPs; rather, they were selected as expert-developed implementations originating from the same CQs. Consequently, the similarity measures evaluate the extent to which the generated ontologies align with expert modelling decisions, while CQ answerability assesses whether they fulfil the original requirements from which both the reference and generated ontologies were derived.

Table 3 presents the semantic similarity between each generated ontology and its corresponding reference ontology, measured using cosine and Jaccard similarity, together with CQ answerability scores for *Stuff* and *AWO*. The results show that ODP guidance had little effect on the overall alignment of the generated ontologies with the reference ontologies. For both *Stuff* and *AWO*, cosine and Jaccard similarity remained unchanged despite the ODP-guided ontologies containing substantially more classes, properties and logical axioms. This indicates that the additional pattern-based modelling neither brought the ontologies closer to nor further away from the reference conceptualisations. Only the *Wine* ontology exhibited a reduction in similarity under ODP guidance, with the cosine similarity decreasing from 0.8 to 0.7 and the Jaccard similarity from 0.4 to 0.2, suggesting that the selected patterns introduced modelling structures that differed from those adopted in the reference ontology.

These results suggest that the LLM captured and represented much of the semantic knowledge required by the CQs, as reflected in the consistently higher cosine similarity scores. In other words, the generated ontologies largely contained the concepts and relations necessary to express the underlying knowledge requirements. However, the comparatively lower Jaccard scores indicate that these semantic requirements were often realised through different modelling structures and axiomatic choices from those adopted in the reference ontologies. Thus, while the generated and reference ontologies remained conceptually similar, they exhibited relatively limited structural overlap, suggesting that ODP guidance influenced how the required knowledge was modelled rather than fundamentally changing what

Table 3

CQ answerability and semantic similarity of the generated ontologies. Semantic similarity is measured against the corresponding reference ontology using cosine and Jaccard Similarity.

Metric	Stuff _{baseline}	Stuff _{ODP}	AWO _{baseline}	AWO _{ODP}	Wine _{baseline}	Wine _{ODP}
CQ Answerability	0.6	0.6	0.4	0.2	N/A	N/A
cosine similarity	0.8	0.8	0.7	0.7	0.8	0.7
Jaccard similarity	0.3	0.3	0.3	0.3	0.4	0.2

knowledge was represented.

CQ answerability exhibited a similar pattern. *Stuff* maintained identical answerability across both conditions, indicating that the additional pattern-based structures neither improved nor impaired the ontology’s ability to satisfy the CQs. In contrast, answerability decreased for *AWO*, suggesting that some of the additional abstractions introduced through ODP-guided modelling may have displaced modelling choices that were more directly aligned with the competency questions. These findings suggest that ODP guidance primarily influences the style of ontology modelling rather than its alignment with an existing expert implementation, which is consistent with our previous findings.

Closer inspection of the cases where CQs were not answered in *AWO* shows that the decrease in CQ answerability is not primarily due to ODP selection, but rather to the way the selected ODPs were instantiated. Most of the recommended ODPs are appropriate for their respective CQs, yet several implementation decisions prevented the resulting ontology from satisfying the queries. For example, *SpeciesEat* is consistently selected for trophic interaction questions such as CQ1: “Which animal eats which other animal?”, *SpeciesHabitat* for habitat-related questions (e.g. CQ11: “Which animals live in this habitat?”) and *Place* for geographical occurrence questions (e.g. CQ9: “Which country is this animal found in?”). In each case, the LLM identifies the recurring modelling problem addressed by the selected pattern. This suggests that the model is capable of recognising the underlying ontology design problem expressed by the CQ. However, the reduction in CQ answerability demonstrates that correct pattern selection alone is insufficient. For example, CQ2 (“Is this animal a herbivore?”) requires no ODP, and the LLM correctly decides not to recommend one. Nevertheless, the generated ontology defines *Herbivore* as an equivalent class without explicitly connecting it to the *Animal* hierarchy in a way that satisfies the evaluation criterion, rendering the CQ unanswerable. This illustrates that modelling errors can arise independently of ODP usage.

A second recurring issue concerns the repeated use of equivalent class definitions where subclass axioms would have better supported retrieval queries. For example, CQ1 (and other *AWO* CQs) all fail for essentially the same reason despite involving different ODPs. In each case, the selected patterns are appropriate, but the generated modelling relies on classes such as *AnimalEatingAnimal*, *AnimalEaten-ByAnimal* and *Predator* being defined as *EquivalentTo* expressions rather than as subclasses suitable for the intended query formulation. This explains why the baseline and ODP-guided ontologies fail on the same CQs: the limitation lies in the LLM’s implementation strategy rather than in the choice of ODP.

As for *Stuff*, both the baseline and ODP-guided ontologies achieved identical CQ answerability (0.6). Since only two CQs were judged by the LLM to require ODPs, the unchanged answerability cannot be explained by incorrect ODP selection. Instead, the remaining failures resulted from the LLM’s implementation of the ontology. For example, CQ7 (“Which stuffs have as part exactly two substuffs?”) failed because the required cardinality constraint was represented as the class *StuffWithExactlyTwoSubstuffs* rather than modelled in a form compatible with the reference query, while CQ10 (“What kind of homogeneous mixture is this stuff?”) remained unanswered due to the absence of the expected modelling of homogeneous mixture kinds.

Overall, the results show that ODP guidance affects multiple dimensions of ontology generation, but not uniformly. Across all benchmarks, ODP-guided generation consistently increased ontology size, logical expressiveness, and the completeness of structural modelling, while reducing several common ontology engineering omissions. However, these improvements were not consistently reflected in

semantic similarity to the reference ontologies or in CQ answerability. The error profile also changed: simple structural omissions became less frequent, whereas more complex conceptual modelling errors became more prevalent. Furthermore, the generated ontologies exhibited different strategies for incorporating ODPs, ranging from direct pattern imports to inline adaptations and unresolved imports, highlighting variability in how the LLM operationalised pattern guidance across domains.

5. Limitations

There are some limitations that should be considered when interpreting the findings of this paper. First, the evaluation was conducted using three ontology benchmarks and a single LLM (GPT-5.5). Although the selected benchmarks represent diverse modelling domains and are widely used in ontology engineering research, a broader validation across additional ontologies, domains, and language models is required to assess the generalisability of the results. Second, each prompting condition was evaluated once per benchmark domain. Given documented prompt sensitivity and output variability in LLM-driven ontology generation [9, 17], the reported differences should be read as observations from a single run rather than statistically validated effects. Repeated generation with variance reporting is left to future work. Third, the ODP-guided and baseline conditions differ not only in the presence of pattern content but in prompt length and explicit scaffolding describing pattern selection, since the ODP library is provided in full. It is therefore not possible to fully separate the effect of pattern content from the effect of increased context length and instructional detail. A control condition using a length-matched but non-pattern-bearing prompt would help isolate this, and is left to future work. Fourth, the three benchmark ontologies (*AWO*, *Stuff*, and *Wine*) and their CQs are established, publicly available resources that may have been included in the model’s pretraining corpus. This raises the possibility that generation performance, particularly in the baseline condition, reflects partial recall of known ontologies rather than construction from the CQs alone. We are unable to rule this out, and note it as a limitation shared with, and previously discussed by, other work in this line [18].

6. Conclusion

This paper investigates the effect of Ontology Design Patterns (ODPs) on Large Language Model (LLM)-based ontology generation from competency questions through a controlled comparison of conventional CQ-based and ODP-guided generation across three benchmark ontologies. The results show that ODP guidance substantially influences the modelling strategies adopted by the LLM, consistently producing larger and structurally richer ontologies while reducing common ontology engineering omissions, particularly missing property domains, ranges, and inverse property declarations. However, these structural improvements do not consistently translate into higher-quality ontologies, as semantic similarity to expert-developed reference ontologies remains largely unchanged and improvements in CQ answerability are inconsistent. Detailed analysis further suggests that these limitations arise primarily from how the LLM instantiates and composes selected ODPs rather than from incorrect pattern selection itself. Collectively, these findings indicate that ODP guidance meaningfully influences the modelling behaviour of LLMs, but that its effectiveness ultimately depends on the model’s ability to consistently operationalise reusable design patterns during ontology generation.

Future work should extend the experimental design by evaluating multiple ontology generations to quantify output variability and introducing length-matched control prompts to isolate the effect of ODP guidance from prompt verbosity. It should also investigate the individual stages of ODP-guided ontology generation, including ODP recognition, selection, and instantiation, to better identify the sources of modelling errors. Finally, developing gold-standard CQ-to-ODP mappings would enable a systematic comparison of LLM-selected patterns against expert recommendations, as well as more fine-grained analyses of ontology quality, including OOPS! severity levels.

References

- [1] M. Fernández-López, A. Gómez-Pérez, N. Juristo, Methontology: from ontological art towards ontological engineering, in: *Proceedings of the AAAI97 Spring Symposium Series on Ontological Engineering*, Stanford, USA, 1997, pp. 33–40.
- [2] M. C. Suárez-Figueroa, A. Gómez-Pérez, M. Fernández-López, The NeOn methodology for ontology engineering, in: *Ontology engineering in a networked world*, Springer, 2011, pp. 9–34.
- [3] V. Presutti, E. Daga, A. Gangemi, E. Blomqvist, eXtreme design with content ontology design patterns, in: *Proc. Workshop on Ontology Patterns*, 2009, pp. 83–97.
- [4] M. Poveda-Villalón, A. Fernández-Izquierdo, M. Fernández-López, R. García-Castro, LOT: An industrial oriented ontology engineering framework, *Engineering Applications of Artificial Intelligence* 111 (2022) 104755.
- [5] N. Tufek, A. S. Thuluva, V. P. Just, F. J. Ekaputra, T. Bandyopadhyay, M. Sabou, A. Hanbury, Validating semantic artifacts with large language models, in: *European Semantic Web Conference*, Springer, 2024, pp. 92–101.
- [6] S. Toro, A. V. Anagnostopoulos, S. M. Bello, K. Blumberg, R. Cameron, L. Carmody, A. D. Diehl, D. M. Dooley, W. D. Duncan, P. Fey, et al., Dynamic retrieval augmented generation of ontologies using artificial intelligence (DRAGON-AI), *Journal of Biomedical Semantics* 15 (2024) 19.
- [7] Y. Rebboud, P. Lisena, L. Tailhardat, R. Troncy, Benchmarking LLM-based ontology conceptualization: A proposal, in: *ISWC 2024, 23rd International Semantic Web Conference*, 2024.
- [8] H. Babaei Giglou, J. D’Souza, S. Auer, LLMs4OL: Large language models for ontology learning, in: *International Semantic Web Conference*, Springer, 2023, pp. 408–427.
- [9] M. J. Saeedizade, E. Blomqvist, Navigating ontology development with Large Language Models, in: *European Semantic Web Conference*, Springer, 2024, pp. 143–161.
- [10] B. Zhang, V. A. Carriero, K. Schreiberhuber, S. Tsaneva, L. S. González, J. Kim, J. de Berardinis, Ontochat: a framework for conversational ontology engineering using language models, in: *European Semantic Web Conference*, Springer, 2024, pp. 102–121.
- [11] S. Tsaneva, S. Vasic, M. Sabou, LLM-driven ontology evaluation: Verifying ontology restrictions with chatgpt., in: *TEXT2KG/DQMLKG@ ESWC*, 2024, p. 15.
- [12] G. Kholmska, K. Kenda, J. Rozanec, Enhancing ontology engineering with LLMs: From search to active learning extensions, *Proceedings of Data Mining and Data Warehouses–Sikdd* (2024).
- [13] D. Garijo, M. Poveda-Villalón, E. Amador-Domínguez, Z. Wang, R. García-Castro, O. Corcho, LLMs for ontology engineering: A landscape of tasks and benchmarking challenges., in: *HGAIS@ ISWC*, 2024.
- [14] V. K. Kommineni, B. König-Ries, S. Samuel, From human experts to machines: An LLM supported approach to ontology and knowledge graph construction, *arXiv preprint arXiv:2403.08345* (2024).
- [15] D. Doumanas, G. Bouchouras, A. Soularidis, K. Kotis, G. Vouros, From human-to LLM-centered collaborative ontology engineering, *Applied Ontology* 19 (2024) 334–367.
- [16] J. Li, D. Garijo, M. Poveda-Villalón, Large language models for ontology engineering: a systematic literature review, *Semantic Web Journal* 20 (2025) 1–45.
- [17] R. Shyama, I. Wilson, A. Ginige, J. Goonetillake, Pattern-based prompting for accurate extraction of ontology assertional (A-box) axioms using LLMs for ontology population, *Procedia Computer Science* 270 (2025) 1438–1447.
- [18] A. S. Lippolis, M. J. Saeedizade, R. Keskiärrkkä, S. Zuppiroli, M. Ceriani, A. Gangemi, E. Blomqvist, A. G. Nuzzolese, Ontology generation using large language models, in: *European Semantic Web Conference*, Springer, 2025, pp. 321–341.
- [19] A. S. Lippolis, M. Ceriani, S. Zuppiroli, A. G. Nuzzolese, Ontogenia: Ontology generation with metacognitive prompting in large language models, in: *European Semantic Web Conference*, Springer, 2024, pp. 259–265.
- [20] D. Doumanas, A. Soularidis, K. Kotis, G. Vouros, Integrating LLMs in the engineering of a SAR ontology, in: *IFIP International Conference on Artificial Intelligence Applications and Innovations*, Springer, 2024, pp. 360–374.

- [21] H. Dong, J. Chen, Y. He, Y. Gao, I. Horrocks, A language model based framework for new concept placement in ontologies, in: *European Semantic Web Conference*, Springer, 2024, pp. 79–99.
- [22] J. H. Caufield, H. Hegde, V. Emonet, N. L. Harris, M. P. Joachimiak, N. Matentzoglou, H. Kim, S. Moxon, J. T. Reese, M. A. Haendel, et al., Structured prompt interrogation and recursive extraction of semantics (SPIRES): A method for populating knowledge bases using zero-shot learning, *Bioinformatics* 40 (2024) btac104.
- [23] A. Gangemi, V. Presutti, Ontology design patterns, in: *Handbook on ontologies*, Springer, 2009, pp. 221–243.
- [24] V. Presutti, E. Blomqvist, E. Daga, A. Gangemi, Pattern-based ontology design, in: *Ontology Engineering in a Networked World*, Springer, 2011, pp. 35–64.
- [25] C. Shimizu, K. Hammar, P. Hitzler, Modular ontology modeling, *Semantic Web* 14 (2023) 459–489.
- [26] M. T. Khan, E. Blomqvist, Ontology design pattern detection-initial method and usage scenarios, in: *SEMAPRO 2010, The Fourth International Conference on Advances in Semantic Processing*, IARIA, 2010, pp. 19–24.
- [27] M. E. Aranguren, E. Antezana, M. Kuiper, R. Stevens, Ontology design patterns for bio-ontologies: a case study on the cell cycle ontology, *BMC bioinformatics* 9 (2008) S1.
- [28] E. Blomqvist, V. Presutti, E. Daga, A. Gangemi, Experimenting with extreme design, in: *International conference on knowledge engineering and knowledge management*, Springer, 2010, pp. 120–134.
- [29] J. M. Mortensen, M. Horridge, M. A. Musen, N. F. Noy, Applications of ontology design patterns in biomedical ontologies, in: *AMIA annual symposium proceedings*, volume 2012, 2012, p. 643.
- [30] P. Mateiu, A. Groza, Ontology engineering with large language models, in: *2023 25th International Symposium on Symbolic and Numeric Algorithms for Scientific Computing (SYNASC)*, IEEE, 2023, pp. 226–229.
- [31] R. M. Bakker, D. L. Di Scala, M. H. de Boer, S. A. Raaijmakers, Ontology learning with LLMs: A benchmark study on axiom identification, *arXiv preprint arXiv:2512.05594* (2025).
- [32] J. Potoniec, D. Wiśniewski, A. Ławrynowicz, C. M. Keet, Dataset of ontology competency questions to SPARQL-OWL queries translations, *Data in brief* 29 (2020) 105098.
- [33] M. Poveda-Villalón, A. Gómez-Pérez, M. C. Suárez-Figueroa, OOPS! (Ontology Pitfall Scanner!): An on-line tool for ontology evaluation, *International Journal on Semantic Web and Information Systems* 10 (2014) 7–34. doi:10.4018/ijswis.2014040102.